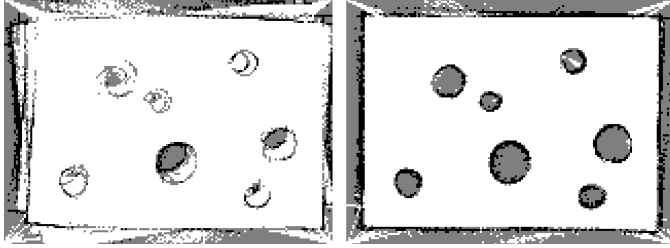


Real-Time Correlative Scan Matching with Sparse Pose Graph Optimization: A Tutorial and Implementation

James Wade

I. INTRODUCTION

Simultaneous Localization and Mapping (SLAM) is the robotic task of globally mapping an unknown environment while simultaneously navigating and localizing within the environment. Ideally, when a robot is deployed in the field, it would be given a map with which to navigate. However, these maps are often not readily available and must be generated as the agent explores. Relying purely on odometry, however, leads to degraded and inconsistent maps due to the accumulation of errors over time, as shown in Figure 1a. As such, full SLAM provides a method of fusing external sensor information (such as LiDAR scans) with odometry to not only improve the robot's current estimate of its pose but also the robot's estimate of its previous trajectory. This trajectory correction leads to a much clearer representation of the environment (Figure 1b) that can then be used for trajectory planning and exploration.



(a) Odometry-only.

(b) SLAM pipeline.

Fig. 1: Occupancy grids constructed from odometry-only poses (left) versus the proposed SLAM pipeline (right).

This paper presents Scan SLAM, a C++ ROS2 package that provides an accessible introduction to and implementation of the full SLAM problem. Scan SLAM combines correlative scan matching [1] and pose graph optimization [2] to provide real-time mapping and localization capabilities. The package is intended not for field-ready deployment, but as a minimal, pedagogical SLAM implementation designed to build intuition for practitioners new to the field. The source code is publicly available here: <https://github.com/James-Wade1/Scan-SLAM>. Popular field-tested SLAM packages include GTSAM's factor-graph SLAM [3] and Steve Macenski's SLAM Toolbox [4] which interfaces well with the popular Navigation2 stack in ROS [5].

II. BACKGROUND

A. Differential Drive Odometry

Odometry provides an estimate of the robot's pose $\mathbf{x} = (x, y, \theta)^T$ by integrating velocity commands over time. For a

differential drive robot, the pose is updated at each timestep dt as

$$x_{t+1} = x_t + v_t \cos(\theta_t) dt \quad (1)$$

$$y_{t+1} = y_t + v_t \sin(\theta_t) dt \quad (2)$$

$$\theta_{t+1} = \theta_t + \omega_t dt \quad (3)$$

where v_t and ω_t are the linear and angular velocities at time t , respectively. In practice, velocity measurements are corrupted by noise, modeled here as zero-mean Gaussian noise added to each command when integrating:

$$\tilde{v}_t = v_t + \mathcal{N}(0, \sigma_v^2), \quad \tilde{\omega}_t = \omega_t + \mathcal{N}(0, \sigma_\omega^2) \quad (4)$$

with $\sigma_v = 0.01$ m/s and $\sigma_\omega = 0.05$ rad/s. Because errors accumulate with each integration step, odometry alone is insufficient for long-horizon navigation, motivating the use of external sensing.

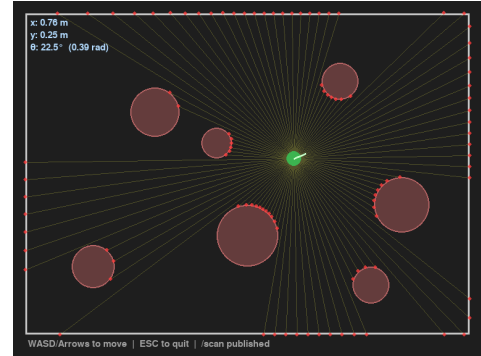


Fig. 2: Differential drive robot simulation with simulated LiDAR.

B. LiDAR

The simulated robot (Figure 2) is equipped with a 360° LiDAR sensor that produces range measurements $\mathbf{s} = \{r_1, \dots, r_N\}$ at uniform angular increments. Each range r_i is obtained via raycasting against the room boundary and circular obstacles in the environment. The resulting scan provides structural information about the environment that can be used to correct accumulated odometry error.

C. Pose Graph SLAM

Graph-based SLAM formulates the localization and mapping problem as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $\mathbf{x}_i \in \mathcal{V}$ represents a robot pose as a snapshot of its trajectory, and each edge $z_{ij} \in \mathcal{E}$ encodes a spatial constraint between poses $\mathbf{x}_i$ and

$\mathbf{x}_j$ with an associated information matrix Ω_{ij} . The information matrix is the inverse of the covariance matrix Σ_{ij} , representing the measure of certainty contained in the constraint z_{ij} . The nodes are also commonly referred to as key frames. The goal of the backend is to find the set of poses $\mathbf{x}^*$ that minimizes the deviation from all constraints:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \sum_{z_{ij} \in \mathcal{E}} \mathbf{e}_{ij}^T \Omega_{ij} \mathbf{e}_{ij} \quad (5)$$

where $\mathbf{e}_{ij}$ is the error between the measured and predicted relative pose between nodes i and j . The construction of edges (the frontend) and the optimization of the graph (the backend) are treated as separate components and are discussed in Sections III-A and III-B, respectively.

III. METHODS

A. Frontend: Correlative Scan Matching

The goal of the SLAM frontend is to generate the pose graph by adding key frames and determining which constraints should be added as edges. Edges connecting non-consecutive nodes are also referred to as loop-closure constraints, as they are typically added when a robot returns to a previously visited region of the environment and recognizes its current scan with previous scans. Adding consecutive or loop-closure constraints follows the same procedure. The full algorithm for correlative scan matching is found in Appendix B.

Given the robot's current scan (candidate scan) and a previous scan (the reference scan), the general process in [1] is as follows: 1) convert the reference scan into a lookup table, 2) transform the candidate scan into the same frame as the reference scan using an initial guess, and 3) perform gradient-free optimization in a search matrix centered around the initial guess, checking combinations of possible $\mathbf{z}_{ij} = (\Delta x, \Delta y, \Delta \theta)$ and choosing the transformation that maximizes some score. The initial guess can be the previous key frame's pose, but it is common to use the odometry-based pose estimate.

1) *Score Definition*: Mathematically, the score can be formulated as the maximum a posteriori (MAP) estimate:

$$\mathbf{z}_{ij}^* = \arg \max_{\mathbf{z}_{ij}} p(\mathbf{s}_j | \mathbf{z}_{ij}, \mathbf{s}_i) p(\mathbf{z}_{ij}), \quad (6)$$

where $\mathbf{s}_i$ and $\mathbf{s}_j$ are the LiDAR scans at key frames i and j , respectively, and $\mathbf{z}_{ij}$ is the spatial constraint/edge that connects key frames i and j . Taking the logarithm of Equation 6,

$$\mathbf{z}_{ij}^* = \arg \max_{\mathbf{z}_{ij}} \underbrace{\log p(\mathbf{s}_j | \mathbf{z}_{ij}, \mathbf{s}_i)}_{\text{scan match likelihood}} + \underbrace{\log p(\mathbf{z}_{ij})}_{\text{odometry prior}}, \quad (7)$$

we see two terms in Equation 7: the odometry prior that acts as a regularizer around our initial estimate of $\mathbf{z}_{ij}$ through the robot's motion model, and the scan match likelihood that measures the likelihood of observing the candidate scan $\mathbf{s}_j$ given the reference scan $\mathbf{s}_i$ and the candidate transformation $\mathbf{z}_{ij}$. We can model both terms as the logarithm of Gaussian distributions, which are reviewed in Appendix A.

The odometry term in Equation 7 can be formulated as a Gaussian around the initial guess $\hat{\mathbf{z}}_{ij}$:

$$\log p(\mathbf{z}_{ij}) \propto -\frac{1}{2}(\mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij})^T \Omega_{\text{pose}}(\mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}), \quad (8)$$

where Ω_{pose} is the propagated inverse of the covariance matrix for the robot's current pose. See [6] for a treatment of first-order covariance propagation.

The scan matching likelihood term can be further simplified by assuming that each individual ray $s^{(k)}$ in the vector of scans $\mathbf{s}$ is independent, allowing the joint probability $p(s_j^{(1)}, s_j^{(2)}, \dots, s_j^{(N)})$ to be decomposed into a product of univariate probabilities, as shown in Equation 9. Taking the log of Equation 9, the product over each entry of $\mathbf{s}_j$ turns into a summation of the logarithms of those univariate probabilities. Therefore, we can compute the scan match likelihood score by the sum of each ray's individual score. We can compute this score efficiently using a lookup table called a likelihood field.

$$p(\mathbf{s}_j | \mathbf{z}_{ij}, \mathbf{s}_i) = p(s_j^{(1)}, s_j^{(2)}, \dots, s_j^{(N)} | \mathbf{z}_{ij}, \mathbf{s}_i) \quad (9)$$

$$= \prod_{k=1}^N p(s_j^{(k)} | \mathbf{z}_{ij}, \mathbf{s}_i). \quad (10)$$

2) *Likelihood Field Construction*: Given a reference scan $\mathbf{s}_i$ at key frame i , a likelihood field $\mathcal{L}$ is constructed by projecting each range measurement into the reference scan's frame and then mapping them onto a discretized grid in locations $\mathbf{p}_i^{(k)} = (p_x^{(k)}, p_y^{(k)})$. Each cell $\mathcal{L}(u, v)$ stores the log-likelihood of another scan point landing at that grid's location in this discretized representation of the world, i.e. $\log p(s_j^{(k)} | \mathbf{z}_{ij}, \mathbf{s}_i)$. Figure 3 (left) shows an example likelihood field, where brighter cells indicate a higher likelihood. Following [1], the value of each cell $\mathcal{L}(u, v)$ is calculated as the sum of Gaussian kernels centered around each range measurement's cell:

$$\mathcal{L}(u, v) = \sum_k \exp \left(-\frac{(u - p_x^{(k)})^2 + (v - p_y^{(k)})^2}{2\sigma_l^2} \right) \quad (11)$$

where σ_l controls the width of each Gaussian kernel and (u, v) are the grid coordinates. In practice, the summation in Equation 11 is dominated by the nearest hit point, since contributions from distant points are negligible. We therefore approximate the sum with a maximum over k , which reduces the construction of $\mathcal{L}$ to a distance matrix encoding the distance to the closest reference scan point, followed by a pointwise log-Gaussian evaluation. This simplified lookup table is much faster to compute than finding the sum of minuscule Gaussian probabilities that far-away grid cells contribute to each other. This field acts as the lookup table for evaluating the likelihood of two scans matching together — given a candidate $\mathbf{z}_{ij}$, the candidate scan $\mathbf{s}_j$ is transformed into the reference frame and each transformed hit point in the discretized space is looked up in $\mathcal{L}$, producing a scalar score:

$$\text{score}(\mathbf{z}_{ij}) = \log p(\mathbf{s}_j | \mathbf{z}_{ij}, \mathbf{s}_i) = \sum_k \mathcal{L} \left(T(\mathbf{z}_{ij}, \mathbf{p}_j^{(k)}) \right) \quad (12)$$

where $T(\mathbf{z}_{ij}, \mathbf{p}_j^{(k)})$ transforms the hit point $\mathbf{p}_j^{(k)}$ by the candidate $\mathbf{z}_{ij}$. The optimal constraint $\mathbf{z}_{ij}^*$ is then the transformation that maximizes this score over the search space.

3) *Branch and Bound*: Finally, an exhaustive search over all possible combinations of $(\Delta x, \Delta y, \Delta \theta)$ is practically infeasible and computationally expensive. This expense can be mitigated by shrinking the bounds of the search space, but at the cost of being limited in how large of a $\mathbf{z}_{ij}$ can be selected. The answer is to implement branch and bound techniques in which we can prune candidate $\mathbf{z}_{ij}$ s that, at best, can give no higher score than our current best constraint candidate. We can perform this by taking a max pool over our likelihood field, as shown in Figure 3. Because we take the maximum over the fine cells to make this coarse representation, the score calculated for each candidate scan in the coarse grid is an optimistic upper bound of its real score in the fine grid. Therefore, we can first perform an exhaustive search within the coarse grid and start evaluating the fine scores within the best coarse grid cells until no coarse grid cell has a higher possible score. This pruning technique eliminates much of the computational time required in the SLAM frontend. The reduction factor needs to balance computational overhead in the coarse search with the tightness of the pruning bound.

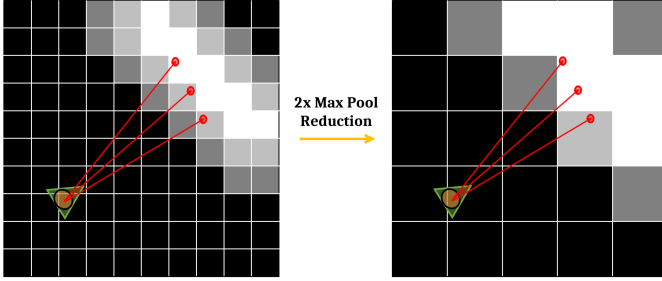


Fig. 3: Illustration of the max-pooling operation used to construct a coarser representation of the score map. The finer resolution map (left) is reduced by a factor of two (right), enabling efficient pruning of the search space via branch-and-bound.

4) *Uncertainty Estimate, Σ_{ij} , and Pose Graph Concatenation*: Once the optimal constraint $\mathbf{z}_{ij}$ has been calculated, the covariance matrix Σ_{ij} also needs to be calculated as it will be used in the backend optimization. The reader is referred to [1] for the comprehensive formulation of calculating Σ_{ij} . However, the method presented in [1] requires an exhaustive search over the fine-resolution likelihood table, so Scan SLAM defaults to using finite difference methods to estimate Σ_{ij} for $p(\mathbf{s}_j | \mathbf{z}_{ij}, \mathbf{s}_i) p(\mathbf{z}_{ij})$. Lastly, we append to the current pose graph the constraint $\mathbf{z}_{ij}$ and the constraint's covariance matrix Σ_{ij} to the set of edges $\mathcal{E}$ and the pose of frame j , $\mathbf{x}_j = \mathbf{x}_i + \mathbf{z}_{ij}$, into the set of nodes $\mathcal{V}$.

5) *Loop-Closures vs Consecutive Scans*: For consecutive scans, choosing the reference scan is intuitive and trivial. However, when checking whether to add a loop-closure constraint, choosing a plausible reference scan is vital to avoid comparing the current scan against all previous ones. Practically, this is done by comparing against key frames whose pose estimates are within some distance threshold. Common distance metrics include the Euclidean distance or the Mahalanobis distance. If a reference key frame is within this threshold, then the MAP posterior is calculated between the candidate scan and

the reference scan, and the resultant $\mathbf{z}_{ij}$ is accepted if MAP estimate exceeds some minimum scoring threshold.

B. Backend: Sparse Pose Graph Optimization

Following many iterations of correlative scan matching, we have constructed a pose graph of key frames $\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_{|\mathcal{V}|}\}$, and constraints $\{(\mathbf{z}_{12}, \Sigma_{12}), (\mathbf{z}_{23}, \Sigma_{23}), \dots\}$. Each constraint $\mathbf{z}_{ij}$ provided by the frontend is modeled as a noisy relative pose measurement between keyframes i and j , together with an information matrix $\Omega_{ij} = \Sigma_{ij}^{-1}$ that encodes the certainty of that measurement. The goal of the backend optimizer is to find the set of poses $\mathbf{x}^*$ that is maximally consistent with *all* the constraints (edges) in the pose graph simultaneously. The algorithm for sparse pose graph optimization is found in Appendix C.

Assuming a Gaussian noise model on our sensor measurements (reviewed in Appendix A), the probability of observing the relative pose constraint given the current pose estimates is

$$p(\mathbf{z}_{ij} | \mathbf{x}_i, \mathbf{x}_j) \propto \exp \left(-\frac{1}{2} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j)^T \Omega_{ij} \mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) \right), \quad (13)$$

where the error function is defined as the difference between the measured relative pose $\mathbf{z}_{ij}$ (computed by correlative scan matching fused with odometry) and the predicted relative pose $\hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$ implied by the current pose estimates:

$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j). \quad (14)$$

Taking the negative logarithm of the Equation 13 yields a quadratic cost term for each edge:

$$-\log p(\mathbf{z}_{ij} | \mathbf{x}_i, \mathbf{x}_j) \propto \frac{1}{2} \underbrace{\mathbf{e}_{ij}^T \Omega_{ij} \mathbf{e}_{ij}}_{\mathbf{F}_{ij}(\mathbf{x}_i, \mathbf{x}_j)}. \quad (15)$$

Assuming the constraints are conditionally independent, the joint negative log-likelihood (up to an additive constant) becomes the sum of the individual cost terms over all edges $\mathcal{E}$:

$$\mathbf{F}(\mathbf{x}) = \sum_{(i,j) \in \mathcal{E}} \mathbf{F}_{ij}(\mathbf{x}_i, \mathbf{x}_j). \quad (16)$$

The optimal set of poses is therefore obtained by minimizing this global cost function:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \mathbf{F}(\mathbf{x}). \quad (17)$$

Equation 16 formulates the pose graph optimization problem as a nonlinear least squares problem. Following the details found in [2], we can express the set of optimal poses as the addition of an initial guess with an optimized change in poses:

$$\mathbf{x}^* = \hat{\mathbf{x}} + \Delta \mathbf{x}^*. \quad (18)$$

While Equation 18 treats the update as ordinary vector addition, pose composition on SE(2) and SE(3) is not strictly additive. For the 2D case used in Scan SLAM, this approximation works well in practice, provided angles are normalized, but

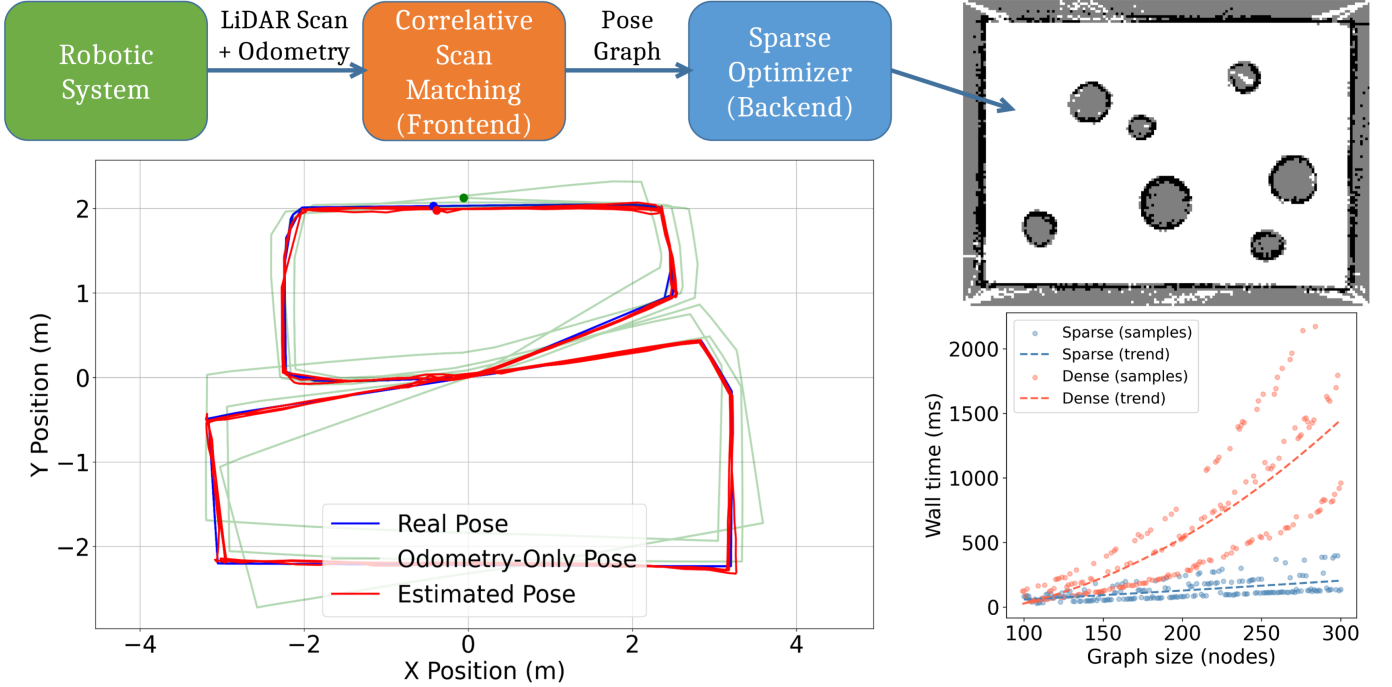


Fig. 4: Overview of the proposed Scan SLAM pipeline. (Top-Left) System architecture showing the frontend correlative scan matching and backend sparse optimizer. (Bottom-Left) Robot trajectory comparison between odometry-only (green), estimated (red), and ground truth (blue) poses. (Right) Resulting occupancy grid map and solver scaling behavior.

3D trajectories on $SE(3)$ require more careful handling via the exponential map. A detailed treatment is found in [2]. Through this reformulation of $\mathbf{x}^*$, the objective can be linearized around $\hat{\mathbf{x}}$ and reduced to the normal equations

$$\mathbf{H}\Delta\mathbf{x}^* = -\mathbf{b}. \quad (19)$$

The above equation can be iteratively solved until a convergence criterion is met. Because $\mathbf{H}$ is typically sparse [2], a sparse solver is far more efficient than a dense one for real-time use.

IV. IMPLEMENTATION

Scan SLAM's implementation closely follows the theory discussed in Sections III-A and III-B. ROS, the Robot Operating System, integrates the simulator, SLAM engine, and various visualization tools through a combination of custom and standard ROS 2 messages. Scan SLAM was written in C++ for the speed of C/C++ over popular alternatives like Python. The sparse solver used is Eigen's SimplicialLDLT sparse cholesky-style solver.

V. RESULTS

Figure 4 showcases the main results of Scan SLAM, namely robot trajectory tracking, runtime analysis, and occupancy grid generation.

A. Pose and Trajectory Error

Figure 4 demonstrates the trajectory correction achieved by Scan SLAM, where the estimated trajectory closely tracks

the ground truth despite the significant drift present in the odometry-only baseline. Figure 5 illustrates the pose graph constructed over the course of the trajectory, with loop-closure constraints highlighted in red. These constraints connect key frames that are spatially proximate but temporally distant, indicating that the frontend successfully recognizes revisited regions of the environment and adds geometrically consistent constraints across the graph.

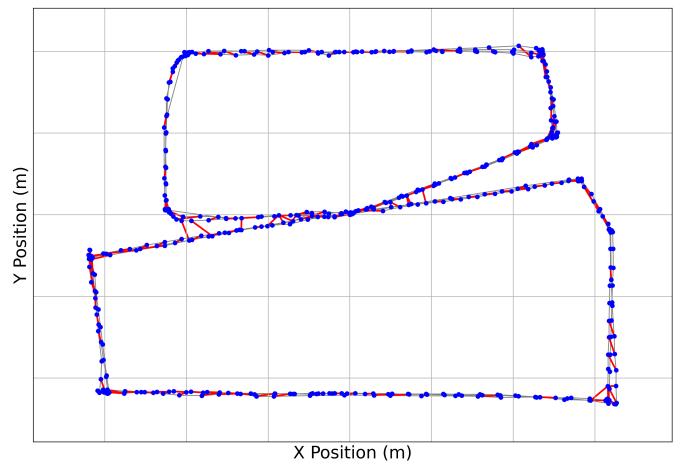


Fig. 5: Generated pose graph for the trajectory traced in Figure 4, with consecutive constraints (blue) and loop-closures (red).

Figure 6 shows the evolution of the robot's pose uncertainty over the course of its trajectory. The first loop closure constraint is added just before 50 seconds, as the robot completes the upper half of the figure-eight path, producing a brief but

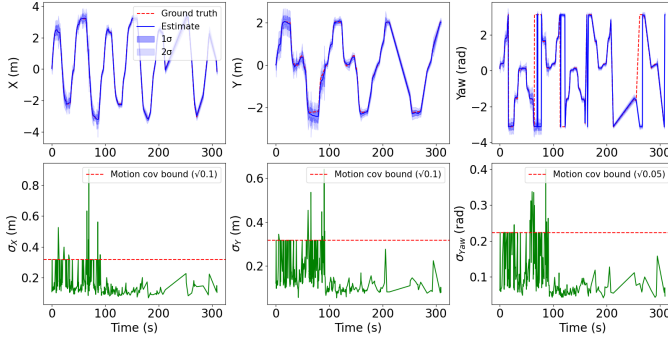


Fig. 6: Evolution of pose uncertainty (σ_x , σ_y , σ_{yaw}) over time (bottom row) alongside the estimated and ground truth trajectories (top row). Vertical reductions in uncertainty correspond to loop closure events, which introduce temporally unrelated constraints into the pose graph and prevent unbounded drift.

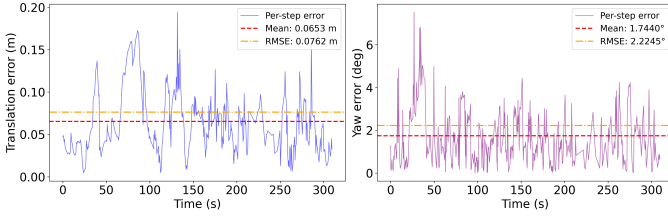


Fig. 7: Absolute trajectory error for tracing the path shown in Figure 4.

visible reduction in uncertainty. As the robot then traverses the lower half, uncertainty grows in the absence of new loop closures. After 100 seconds, as the robot begins revisiting previously explored regions, loop closures are added more frequently and the uncertainty stabilizes at a consistently low level for the remainder of the run. This behavior illustrates the core role of loop closure constraints in pose graph SLAM: by introducing long-range consistency into the graph, they prevent unbounded drift and anchor the trajectory to a globally consistent solution. Additionally, the red dotted lines indicate the motion covariance added through the motion model when performing consecutive scan matching. By fusing LiDAR data to correct the prior odometry-based belief of the robot's pose, we should expect covariances that are at least no worse than the prior, and Figure 6 shows that this is generally true for Scan SLAM.

B. Runtime and Iterations

Figure 8 shows the runtime summary for Scan SLAM. While an average of 2547.6 iterations for the addition of a loop closure might seem excessive, the vast majority of those iterations were in evaluating the coarse search grid described in Figure 3. For Scan SLAM's default search grid, without pruning techniques, over 35,000 iterations would have been required. Experimentation with the reduction factor from the fine resolution grid to the coarse grid can be performed to find more optimal hyperparameters.

Additionally, the right-hand side of Figure 4 shows the effect of graph size on the solution time for a dense solver

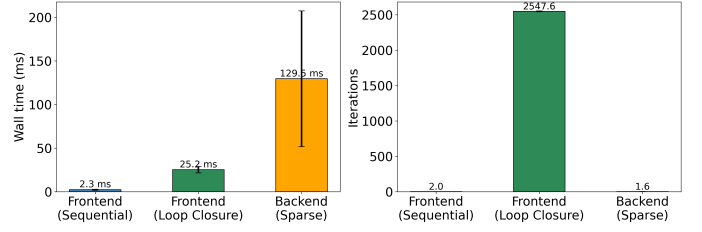


Fig. 8: Runtime summary for the frontend and the backend of Scan SLAM. The black bar indicates the standard deviation of the empirical data. The frontend is faster than one might expect given its search-space size, thanks to branch-and-bound pruning; the backend dominates runtime as the graph grows.

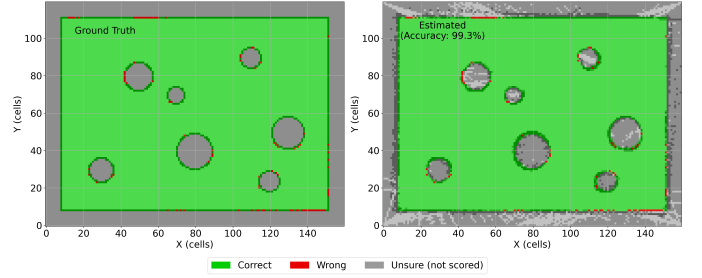


Fig. 9: (Left) Ground truth occupancy grid for the simulator shown in Figure 2. (Right) A pixel-by-pixel comparison of the generated map, with a one-pixel tolerance, achieving an accuracy of 99.3%

and a sparse solver. These results clearly indicate the efficiency boost of using a sparse solver, following from the fact that the left-hand side of Equation 19 is generally very sparse.

C. Occupancy Grid Generation

The upper right-hand side of Figure 4 shows the generated occupancy grid for the entire map. Figure 9 shows the pixel-by-pixel match between the ground truth occupancy grid and Scan SLAM's generated grid, allowing for a one-pixel tolerance. Even without the tolerance, the map still achieves an accuracy of 95%. Thus, I am confident in Scan SLAM's ability to map its environment with a high degree of accuracy.

VI. CONCLUSION

Scan SLAM has shown solid initial performance in a simulated environment with injected noise. The goal of this package is not for field deployment but rather as a learning tool with a low learning curve for those interested in learning more about SLAM. Future work will include exploring different methods of scan matching (such as Iterative Closest Point and Iterative Closest Line), the effects of different sparse solvers, and the eventual integration of path planning using the generated occupancy grid.

REFERENCES

- [1] E. B. Olson, "Real-time correlative scan matching," in *2009 IEEE international conference on robotics and automation*. IEEE, 2009, pp. 4387–4393.

- [2] G. Grisetti, R. Kümmerle, C. Stachniss, and W. Burgard, “A tutorial on graph-based slam,” *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, 2011.
- [3] F. Dellaert, M. Kaess *et al.*, “Factor graphs for robot perception,” *Foundations and Trends® in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017.
- [4] S. Macenski and I. Jambrecic, “Slam toolbox: Slam for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021. [Online]. Available: <https://doi.org/10.21105/joss.02783>
- [5] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, “The marathon 2: A navigation system,” in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. [Online]. Available: <https://github.com/ros-planning/navigation2>
- [6] S. Thrun, “Probabilistic robotics,” *Communications of the ACM*, vol. 45, no. 3, pp. 52–57, 2002.

APPENDIX A

GAUSSIAN DISTRIBUTION

A univariate Gaussian distribution over a random variable x is defined by its mean μ and variance σ^2 , where σ is the standard deviation describing the spread of the distribution:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \quad (20)$$

Taking the logarithm, the additive constant terms can be dropped when optimizing over $\mathbf{x}$, leaving:

$$\log p(x) \propto -\frac{(x-\mu)^2}{2\sigma^2} \quad (21)$$

For a multivariate Gaussian over a vector $\mathbf{x} \in \mathbb{R}^n$, the distribution is parameterized by a mean vector $\boldsymbol{\mu}$ and a covariance matrix $\boldsymbol{\Sigma} \in \mathbb{R}^{n \times n}$, where the diagonal entries of $\boldsymbol{\Sigma}$ represent the variance of each dimension and the off-diagonal entries encode correlations between dimensions:

$$p(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^n |\boldsymbol{\Sigma}|}} \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right) \quad (22)$$

where $|\boldsymbol{\Sigma}|$ is the determinant of the covariance matrix and $\boldsymbol{\Sigma}^{-1} = \boldsymbol{\Omega}$ is the information matrix. Taking the logarithm and discarding the terms independent of $\mathbf{x}$:

$$\log p(\mathbf{x}) \propto -\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Omega}(\mathbf{x}-\boldsymbol{\mu}) \quad (23)$$

This log-likelihood form is central to both the frontend and backend of the SLAM pipeline, as maximizing the log-likelihood is equivalent to minimizing a weighted least-squares error, which is the form solved by the sparse pose graph optimizer.

APPENDIX B

ALGORITHM FOR CORRELATIVE SCAN MATCHING

Algorithm 1 implements correlative scan matching, following the implementation described in [1]. The algorithm utilizes a max-heap in which the candidate $\mathbf{z}_{ij}$ with the highest score is removed from the queue first.

APPENDIX C

ALGORITHM FOR SPARSE POSE GRAPH OPTIMIZATION

Comprehensive pseudocode for the sparse pose graph optimization algorithm can be found in [2].

Algorithm 1 Correlative Scan Matching

Require: candidate scan $\mathbf{s}_j$, reference scan $\mathbf{s}_i$, odometry seed $\hat{\mathbf{z}}_{ij}$

Ensure: optimal transformation $\mathbf{z}_{ij}^*$, information matrix $\boldsymbol{\Omega}_{ij}$

- 1: Build fine-resolution likelihood field $\mathcal{L}_{\text{fine}}$ from reference scan $\mathbf{s}_i$
- 2: Construct $\mathcal{L}_{\text{coarse}}$ via max-pooling of $\mathcal{L}_{\text{fine}}$
- 3: Initialize max-heap priority queue $\mathcal{Q}$
- 4: Initialize best score $s^* \leftarrow -\infty$, $\mathbf{z}_{ij}^* \leftarrow \emptyset$
- 5: **for** each candidate $\mathbf{z}_{ij}$ in coarse search space centered on $\hat{\mathbf{z}}_{ij}$ **do**
- 6: Compute upper-bound score $\bar{s} \leftarrow \text{score}(\mathbf{z}_{ij}, \mathcal{L}_{\text{coarse}})$
- 7: Push $(\bar{s}, \mathbf{z}_{ij})$ onto $\mathcal{Q}$
- 8: **end for**
- 9: **while** $\mathcal{Q}$ is not empty **do**
- 10: Pop $(\bar{s}, \mathbf{z}_{ij})$ from $\mathcal{Q}$
- 11: **if** $\bar{s} < s^*$ **then**
- 12: **break** ▷ All remaining candidates are suboptimal
- 13: **end if**
- 14: **for** each fine candidate $\mathbf{z}'_{ij}$ within the coarse cell of $\mathbf{z}_{ij}$ **do**
- 15: Compute fine score $s \leftarrow \text{score}(\mathbf{z}'_{ij}, \mathcal{L}_{\text{fine}})$
- 16: **if** $s > s^*$ **then**
- 17: $s^* \leftarrow s$, $\mathbf{z}_{ij}^* \leftarrow \mathbf{z}'_{ij}$
- 18: **end if**
- 19: **end for**
- 20: **end while**
- 21: Estimate $\boldsymbol{\Omega}_{ij}$ via finite differences of score around $\mathbf{z}_{ij}^*$
- 22: **return** $\mathbf{z}_{ij}^*$, $\boldsymbol{\Omega}_{ij}$
